RETAILSPHERE (PTY) LTD

Data Platform Programme

Testing Strategy

Step 12 Deliverable — Testing Strategy

Document ID	TST-001
Version	**1.0 - Approved**
Date	22 October 2026
Author	Thozamile Mbalo – Data Solutions Architect
Reviewed by	Nomvula Mahlangu – Chief Data Officer
Target Platform/Tooling	**dbt tests/ Azure Data Factory / Microsoft Fabric**
Status	**Approved**

1. Purpose

This document defines RetailSphere's testing approach across unit, integration, reconciliation, and business validation testing. It satisfies FR-CCR-007 — no bad data may silently reach the Gold layer — and formalizes the testing of the automated documentation-completeness gate committed to in MET-001 Section 6, which must be proven working during this step, before Step 14's first Silver model is built.

2. Scope

TST-001 defines *what* must be tested, *at what layer*, and *by whom* — it does not itself contain the test code. The documentation gate configuration is built in Step 12; the dbt test implementations and reconciliation scripts are Step 13+ build artifacts. This mirrors the standard-versus-artifact boundary already established in MET-001: this document is the standard those build-phase artifacts must meet.

3. Data Quality Testing

Tests are layered by what each layer is responsible for. Each layer tests only what it owns.

Bronze — arrival checks, not data-quality checks. Bronze holds raw data as received, which can legitimately contain duplicates (e.g. multiple extracts of one purchase order across its lifecycle) and nulls (cleaned in Silver). Uniqueness and not-null tests are therefore not applied here. Bronze tests confirm the data *arrived properly*:

- **Freshness:** data landed within its expected schedule (dbt **source freshness**)
- **Volume:** row count above zero and not drastically below the expected range
- **Schema:** expected columns present, catching silent structural changes at source

All three run against the Bronze landing tables' own load metadata. No direct source-system access is required.

Silver — structural tests. The generic dbt tests mapped in DID-001 Section 4 (not_null, unique, composite unique, relationships, accepted_values) enforce every constraint declared informational-only in PDD-001 Section 5.

Silver to Gold — business-rule tests. Rules no generic test can express are enforced as custom dbt tests at the Silver-to-Gold boundary. Examples: **debit_amount** and **credit_amount** never both non-zero on one FactFinance line; **contract_end_date** never before contract start in FactSupplierContract. Rules are sourced from PDD-001's table notes, DM-001's column definitions, and FRD-001's acceptance criteria, and are written as each model is built, following the same incremental discipline MET-001 applies to documentation.

Severity — v1.0: every test halts and alerts. Consistent with FR-CCR-007, any test failure halts the affected domain's pipeline and alerts per DID-001 Section 6. Domain isolation (DID-001 Section 7) limits the impact to one domain. This also covers upstream changes nobody communicated, such as a new marketing channel value arriving unannounced: the halt is the safety net when the heads-up doesn't come. A tiered warn/error model is deferred until real failure patterns are observed during Steps 14 and 15, and will be agreed with domain owners rather than decided in advance.

4. Unit Testing

Section 3 tests whether *data* is correct on every run. Unit tests test whether the *logic* is correct, using small synthetic inputs where the right answer is already known. They catch the failure that data tests cannot: logic that is wrong but produces plausible output, such as the wrong customer linked to the right-looking record.

Tooling and limits. dbt native unit tests (dbt 1.8+) support SQL models only, not snapshots, seeds or sources. They run in development and CI, not in production, since fixed inputs add nothing on live runs. All test inputs are synthetic, consistent with MET-001's rule on sample data. Fabric adapter support is to be verified during initial dbt setup.

Identity resolution (STM-001 section 4.3, ADR-002)

Case	Input	Expected result
1	Same ID number, name spelled differently	Matched, tier 1 (deterministic)
2	ID number missing on one or both records; same first name + surname + phone	Matched, high confidence
3	Same email + first name only	Matched, medium confidence
4	Same phone + first name only	Low confidence, flagged for human validation
5	Different ID numbers; same name and phone	Not merged; flagged for review

Case 5 is a proposed rule not specified in STM-001: conflicting ID numbers are treated as evidence of different people, since wrongly merging two customers' contact history creates POPIA risk (e.g. an opt-out landing on the wrong person). Pending ADR-002 sign-off.

SCD Type 2 (six dimensions). dbt snapshots cannot be unit tested natively, and their internal logic is dbt's own. The risk is configuration, so it is tested by running synthetic data through the snapshot in the dev environment: changing a watched attribute must close the old row and create a new current row; changing an unwatched column must create no new row. These run as ordinary dbt tests.

Duration calculations in the accumulating snapshots (**days_to_pick**, **days_to_dispatch**, **days_to_receipt**, **total_days_pipeline** and equivalents): correct day counts, and NULL where a milestone date is not yet populated.

Contact-count logic (FactContactHistory), where computed in a dbt model: the 4-contact monthly cap is counted correctly, including suppressed attempts, at month boundaries.

Out of scope for unit tests. Opt-out propagation ("success means all four systems confirmed") lives in the ADF pipeline, not a SQL model, so it is tested end-to-end with Priya's team (Section 7). LLM-based PDF extraction is Python, not SQL, so it is covered by the spot-check process flagged in DID-001.

5. Reconciliation Testing

DID-001 Section 7 defines the reconciliation controls that run on every load. This section tests that those controls actually detect errors and halt the pipeline. A control that has never been seen to fail is unproven.

Principle. Defects are planted *between* the layers being compared, never upstream of the baseline. An upstream defect is inherited by every layer and passes reconciliation, so it belongs to the data tests in Section 3.

Three legs are tested:

- **Source to Bronze**: in test, Bronze is compared to the generator’s answer key (independent counts and control totals, computed by separate code). In production, the source-side control total required by FR-FIN-002 is the baseline.

- **Bronze to Silver**: Bronze count = Silver count + rejected rows. Rejected rows are logged to a reject table with a reason and never dropped silently.
- **Silver to Gold**: the method varies by table type, per DID-001 Section 7.

#	Table type	Planted defect	Check that must fail
1	Insert-only	Remove one row between Silver and Gold	Row count
2	Monetary	Alter one amount in Gold only	Control total (row count still matches)
3	Accumulating snapshot	Leave Gold on an earlier stage than Silver	Latest-extract comparison
4	Silver	Remove a row without logging it as rejected	Bronze = Silver + rejects
5	Source to Bronze	Drop one row during Bronze load	Answer-key comparison
6	Any	Run on clean data	Nothing fails (no false alarms)

Pass criteria. Every planted defect is detected, halts only the affected domain's Gold load, and raises an alert per DID-001 Section 6. Case 6 passes with zero variance. A deliberate failure in one domain must leave the other five loading normally.

Dependencies. The generator must produce the answer key (Python deliverable). The reject-table design is a Step 14 build artifact. The production source-side control-total method per source is a Step 13 design detail.

6. Documentation Gate Testing

MET-001 Section 6 commits RetailSphere to an automated documentation-completeness gate, proven working here in Step 12 before the first Silver model is built in Step 14. As in Section 5, each test plants a defect and confirms the gate fails.

Enforcement. The gate runs as a required CI check on pull requests to main, with branch protection applied to administrators. A pre-commit hook may give early feedback but is not the gate, since it cannot be enforced on every machine. Tool selection and Fabric adapter support are confirmed during setup.

#	Planted defect	Expected results
1	Model with no description	Fail
2	Declared column with no description	Fail
3	Column in the SQL but not declared in YAML	Fail (needs an all-columns check)
4	Description deleted after an earlier build	Fail (CI re-parses the project first)
5	Placeholder description (“TBD”)	Fail (custom check)
6	Same column, different descriptions in two models	Fail
7	Fully documented model	Pass, no false alarm
8	Direct push to main, bypassing the PR	Blocked, including for the repository owner

Pass criteria. Every planted defect fails as expected, case 7 passes, and case 8 proves the gate cannot be skipped.

Known limit. The gate proves documentation exists and is consistent, not that it is true. Accuracy stays with MET-001’s quarterly audit and domain-owner review.

Dependencies. A small throwaway dbt project with synthetic models. Description checks read dbt’s manifest and YAML files, while column checks also need catalog metadata, so cases 1, 2 and 4-7 can run on a local setup. Case 3 and all Fabric-specific behaviour must be confirmed against Fabric.

7. Business/UAT Validation

UAT confirms that what was built meets what was agreed. It is the business's acceptance, not the engineering team's.

Who validates what.

Requirements	Validated by	Go-live sign-off
Domain requirements (FR-PRO, FR-INV, FR-SAL, FR-CUS, FR-FIN, FR-MKT)	The domain's owner, against each requirement's FRD-001 acceptance criteria	Domain owner and CDO
FR-CCR-005 (reconciliation)	Johan van der Merwe	CDO
FR-CCR-004, FR-CCR-008 (POPIA)	Priya Naidoo	CDO
FR-CCR-001 to 003, 006, 007	CDO (single signer by design; no business owner depends on these directly)	CDO

The Lead Data Engineer demonstrates requirements during UAT but never validates them, keeping validation independent of the build.

When. UAT runs per domain as each is built, not once at the end of the build, starting with Sales. Problems surface while they are cheap to fix, and stakeholders see the platform incrementally, not for the first time all at once.

Data. Build and test environments use synthetic or masked data, never raw production personal data. UAT runs in a controlled environment where owners can compare results against figures they already trust; for domains holding personal information, access is restricted per MET-001's POPIA tiers and subject to ADR-002 and ADR-007 sign-off. *For RetailSphere as a portfolio project, synthetic data with the generator's answer key stands in for real data. This tests correctness but not business recognition, which is a stated limitation.*

Rejection. An owner may reject a requirement with a reason and a preferred outcome. Rejections are logged as defects, fixed and re-tested.

Go-live. A domain goes live only with sign-off from both its domain owner and the CDO. A known defect can be accepted only as a documented exception, with a reason, an owner and a fix-by date, so exceptions don't quietly become the standard.

8. Test Environment and Data

Environments. Three environments, plus CI as an automated runner:

Environment	Platform	Data	Purpose
Dev	Local DuckDB	Synthetic	Building models, unit tests, planted-defect testing
Test\UAT	Fabric workspace	Synthetic or masked, per section 7	Integration on the real platform, live reconciliation, business UAT
Production	Fabric workspace	Real (portfolio: synthetic)	Live platform, built at Step 18
CI (runner)	GitHub Actions	Synthetic	Documentation gate and unit tests on every pull request

The same dbt project serves all environments through dbt targets; no code changes between them.

Which tests run where.

Test type	Dev	CI	Test\UAT	Production
Data tests sec.3	Yes	Yes	Yes	Yes

Unit tests sec.4	Yes	Yes	No	No
Reconciliation, planted defects sec.5	Yes	No	Yes, proving run only	No
Reconciliation, live controls (DID-001 sec.7)	No	No	Yes	Yes
Documentation gate sec.6	Optional	Yes (required)	No	No
Business UAT sec.7	No	No	Yes	No

Apart from the single proving run in a separate schema, planted defects never run outside dev, so the environment domain owners use stays trustworthy.

Unverified. Fabric-specific behaviour (assumptions register rows 4-9) and Fabric capacity cost are to be confirmed before the Test/UAT workspace is built.

Proving run. Before UAT opens, the Section 5 planted-defect tests run once on Fabric in a separate schema from the one domain owners use, proving the controls work on the production platform, not only on DuckDB. A clean run (Section 5, case 6) follows to confirm no false alarms before owners are given access.

9. Next Steps

The completion of TST-001 marks the end of Step 12's design work. Step 12 itself closes only once the documentation gate has run as a required CI check (Section 6), since MET-001 commits to proving it in this step.

Immediate:

- CDO review of TST-001, then publish it and update the Notion tracker and README.
- Build the GitHub Actions workflow and branch protection, and run the Section 6 test cases.

Outstanding items before or during build:

- **Assumptions register:** rows 4-9 and 12-14 to be verified, mostly against Fabric. Corrections raised to affected documents as needed.
- **Synthetic data generator** (Python), including the independent answer key Section 5 depends on.
- **Reject table design** for Bronze-to-Silver reconciliation (Step 14), and the production source-side control-total method per source (Step 13).
- **Identity resolution case 5** (conflicting ID numbers not merged): a proposed rule pending ADR-002 sign-off.
- **Test severity tiers:** all tests halt in v1.0; a warn/error model is to be agreed with domain owners once real failure patterns appear.
- **Fabric capacity cost:** to be confirmed before the Test/UAT workspace is built
- **Carried forward:** ADR-002, 003, 006 and 007 sign-offs; FactContactHistory attempt-sequencing validation; DID-001 v1.1 CDO sign-off; MET-001 v1.1 wording correction; a short ADR recording why dbt was chosen over ADF data flows.

10. Document Change Log

Version	Date	Author	Description
1.0 - Draft	22 October 2026	Thozamile Mbalo	Initial TST-001 draft. Defines layered testing: Bronze arrival checks, Silver structural data tests, and Silver-to-Gold business-rule tests, all halting on failure in v1.0 (Section 3, Data Quality Testing). Adds targeted unit tests for logic that can be wrong yet look valid: identity resolution, SCD Type 2 configuration, duration calculations and contact counts (Section 4). Reconciliation testing proves controls catch planted defects across three legs, using a generator answer key (Section 5). Documentation gate testing defines eight planted cases with CI enforcement via branch protection (Section 6). Business UAT runs per domain with defined validators, rejection handling and dual sign-off for go-live (Section 7). Three environments plus CI, with a test-by-environment matrix (Section 8). Found and raised DID-001 v1.1 (reconciliation gap at Silver) and MET-001 v1.1 (gate wording).
1.0 - Approved	22 October 2026	Reviewed by Nomvula Mahlangu, Chief Data Officer	Approved following review. Section 7: one owner per requirement, with FR-CCR-005 validated by Johan and the Lead Data Engineer excluded from validation to keep it independent of the build. Section 8: planted-defect proving run added in a separate Fabric schema before UAT opens, since controls proven on DuckDB are not proven on Fabric.

— End of Document —